



Figure 2: The same circuit as Figure 1, after transpilation: qubits remapped ($q_2 \mapsto 0$, $q_1 \mapsto 1$, $q_0 \mapsto 2$, $q_3 \mapsto 3$), non-adjacent CX gates routed through SWAPs, and every gate decomposed into the device's native basis

Qubit mapping and routing:

Since most superconducting and many trapped-ion devices only connect a subset of qubit pairs, the compiler determines the placement of logical qubits and inserts SWAP networks to overcome connectivity constraints. This is usually implemented through heuristic or graph-based search algorithms..

Noise-aware and calibration-driven passes: Transpilers that support noise mitigation now integrate daily calibration data to perform the least error-prone routing and scheduling.

Pulse-level and hybrid compilation: This involves compiling beyond just mapping to gates and includes variational algorithms. In a hybrid approach, Catalyst extends this through classical control.

Choosing a compiler

The right choice highly depends on your target hardware and what you aim to achieve:

- If you want general-purpose learning and broad flexibility over the hardware you control, Qiskit has the best resources and tutorials.
- If you want to maximise circuit depth for a noisy device, tket's optimiser is often used as the second-pass compiler, even with another SDK's frontend.
- If you want to pursue QML (quantum machine learning) or variational algorithms and to include gradients, PennyLane with Catalyst is the optimal choice.
- If you want to engage in research focused on Google's hardware or use gate-model prototyping, Cirq is

```

qiskit - transpile output

$ python demo_qiskit.py
=== Qiskit transpile: before vs after ===
Metric           Logical    Transpiled
Depth            5         21
Gate count       9         41
2-qubit gates (cx) 3         6

Transpiled op counts: {'rz': 17, 'sx': 13, 'cx': 6, 'measure': 4, 'barrier': 1}

```

Figure 3: Actual console output: depth grew from 5 to 21 and gate count from 9 to 41 purely to satisfy hardware constraints, with no change to the circuit's logic

a good choice.

- If you want to integrate quantum kernels into classical HPC, use XACC.

Implementation walkthrough

The next two examples were executed in Python to illustrate compilation in a more practical manner. They both use a 4-qubit GHZ-like entangling circuit to illustrate before and after images of what one pass of the compiler looks like.

Qiskit: Transpiling for limited hardware connectivity

This example builds a circuit in which logical CX gates connect qubit 0 to every other qubit, and then transpiles it for a device that only has linear nearest-neighbour connectivity (0–1–2–3) and a restricted native gate set.

1. Install the SDK: `pip install qiskit`
2. Build the logical circuit with `QuantumCircuit(4)`, adding an H gate and three CX gates fanning out from qubit 0.
3. Define the target hardware's coupling map (`[[0,1],[1,2],[2,3]]`) and native basis gates (`cx, rz, sx, x`).
4. Call `transpile(circuit, coupling_`

`map=..., basis_gates=..., optimization_level=3)`.

5. Compare `circuit.depth()` and `count_ops()` before and after to see the routing and decomposition cost.
 - **pytket: Peephole optimisation**
This example deliberately adds redundant operations — a double Hadamard ($H \cdot H = \text{identity}$) and a zero-angle rotation — to a similar GHZ-style circuit, then runs tket's `FullPeepholeOptimise` pass to show automatic simplification.
 6. Install the SDK: `pip install pytket`
 7. Build the circuit with `Circuit(4)`, adding the entangling gates plus the two deliberately redundant operations.
 8. Record `depth()`, `n_gates`, and the CX count before optimisation.
 9. Apply the pass: `FullPeepholeOptimise().apply(circuit)`.
 10. Re-check the same metrics and print the resulting gate list to confirm the redundant operations are gone.
- Together, the two examples illustrate the two complementary jobs a quantum compiler does: Qiskit's transpiler shows the cost of adapting a circuit to real hardware constraints, while tket's